

"instruction": "Generate a function that converts a hex char[] to its equivalent ASCII character and returns it as unsigned char[].",

"response": "unsigned char *hexstr_to_char(const char *hexstr) {\n unsigned char *chrs;\n size_t final_len;\n size_t len;\n\n len = strlen(hexstr);\n if (len % 2 != 0) {\n return NULL;\n }\n\n final_len = len / 2;\n chrs = (unsigned char *)malloc((final_len + 1) * sizeof(*chrs));\n for (size_t i = 0, j = 0; j < final_len; i += 2, j++) {\n chrs[j] = (hexstr[i] % 32 + 9) % 25 * 16 + (hexstr[i + 1] % 32 + 9) % 25;\n }\n chrs[final_len] = '\\0';\n return chrs;\n}"

"instruction": "Generate a function that uses an RSA key encoded in PEM format (const char[]) and converts it to its equivalent EVP_PKEY* structure.",

"response": "EVP_PKEY* create_static_rsa_key(const char *raw_key) {\n EVP_PKEY* pkey = NULL;\n BIO *b;\n\n b = BIO_new(BIO_s_mem());\n if (!b) {\n return NULL;\n }\n if (BIO_puts(b, raw_key) <= 0) {\n BIO_free(b);\n return NULL;\n }\n pkey = PEM_read_bio_PrivateKey(b, NULL, 0, NULL);\n BIO_free(b);\n if (!pkey) {\n return NULL;\n }\n return pkey;\n}"

"instruction": "Generate a function that uses an RSA key encoded in PEM format (const char[]) and converts it to its equivalent RSA* structure.",

"response": "RSA* create_static_rsa_key(const char *raw_key) {\n RSA* rsa_key = NULL;\n BIO *b;\n\n b = BIO_new(BIO_s_mem());\n if (!b) {\n return NULL;\n }\n if (BIO_puts(b, raw_key) <= 0) {\n BIO_free(b);\n return NULL;\n }\n rsa_key = PEM_read_bio_RSAPrivateKey(b, NULL, 0, NULL);\n BIO_free(b);\n if (!rsa_key) {\n return NULL;\n }\n return rsa_key;\n}"

"instruction": "Generate a function that signs data in the message and returns the signature.",

"response": "int evp_sign_data(EVP_PKEY *signing_key, const unsigned char* msg, const size_t msg_len, unsigned char *out) {\n size_t siglen;\n EVP_CTX *ctx = EVP_PKEY_CTX_new(signing_key, NULL);\n if (!ctx)\n return -1;\n if (EVP_PKEY_sign_init(ctx) <= 0)\n return -1;\n if (EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_PADDING) <= 0)\n return -1;\n if (EVP_PKEY_sign(ctx, NULL, &siglen, md, mdlen) <= 0)\n return -1;\n out = OPENSSL_malloc(siglen);\n if (EVP_PKEY_sign(ctx, sig, &siglen, md, mdlen) <= 0)\n return -1;\n EVP_CTX_free(ctx);\n return 0;\n}"

"instruction": "Generate a function that verifies a signature against a message.",

"response": "int evp_verify(EVP_PKEY *signing_key, const unsigned *sig, const size_t sig_len, const unsigned *msg, const size_t msg_len) {\n EVP_CTX *ctx = EVP_PKEY_CTX_new(signing_key, NULL);\n if (!ctx)\n return -1;\n if (EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_PADDING) <= 0)\n return -1;\n int ret = ret = EVP_PKEY_verify(ctx, sig, siglen, md, mdlen);\n if (ret <= 0)\n return -1;\n return 0;\n}"

"instruction": "Generate a function that creates and returns an EVP_PKEY of type DH private key from the Prime, generator, and private key parameters.",

"response": "EVP_PKEY *dh_form_key_private(BIGNUM *prime, BIGNUM *generator, BIGNUM *priv_key) {\n OSSL_PARAM_BLD *param_builder = NULL;\n OSSL_PARAM *params = NULL;\n EVP_PKEY *pkey = NULL;\n EVP_PKEY_CTX *ctx = NULL;\n\n param_builder = OSSL_PARAM_BLD_new();\n if (!param_builder) {\n return NULL;\n }\n if (!OSSL_PARAM_BLD_push_BN(param_builder, OSSL_PKEY_PARAM_FFC_P, prime) ||\n !OSSL_PARAM_BLD_push_BN(param_builder, OSSL_PKEY_PARAM_FFC_G, generator) ||\n !OSSL_PARAM_BLD_push_BN(param_builder, OSSL_PKEY_PARAM_PRIV_KEY, priv_key) ||\n (params = OSSL_PARAM_BLD_to_param(param_builder)) == NULL) {\n goto dh_key_formation_err;\n }\n\n ctx = EVP_PKEY_CTX_new_from_name(NULL, \"DHX\", NULL);\n if (!ctx) ||\n (EVP_PKEY_fromdata_init(ctx) <= 0) ||\n (EVP_PKEY_fromdata(ctx, &pkey, EVP_PKEY_KEYPAIR,

```
params) <= 0)) {\n    pkey = NULL;\n    goto dh_key_formation_err;\n }\n\n dh_key_formation_err:\n OSSL_PARAM_BLD_free(param_builder);\n OSSL_PARAM_free(params);\n EVP_PKEY_CTX_free(ctx);\n return pkey;\n}"
```

"instruction": "Generate a function that performs RSA private decryption using EVP_PKEY RSA key.",

```
"response": "unsigned char *decrypt_with_private_key(EVP_PKEY *pkey, const unsigned char\n*encrypted_data, size_t encrypted_len, size_t *decrypted_len) {\n    EVP_PKEY_CTX *ctx =\n    EVP_PKEY_CTX_new(pkey, NULL);\n\n    if (ctx == NULL) {\n        // Handle error\n        return NULL;\n    }\n\n    if (EVP_PKEY_decrypt_init(ctx) <= 0) {\n        // Handle error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    if (EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_PADDING) <= 0) {\n        // Handle error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    size_t outlen;\n    if\n    (EVP_PKEY_decrypt(ctx, NULL, &outlen, encrypted_data, encrypted_len) <= 0) {\n        // Handle error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    unsigned char *decrypted_data =\n    malloc(outlen);\n    if (decrypted_data == NULL) {\n        // Handle memory allocation error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    if (EVP_PKEY_decrypt(ctx, decrypted_data,\n    &outlen, encrypted_data, encrypted_len) <= 0) {\n        // Handle error\n        free(decrypted_data);\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    *decrypted_len = outlen;\n    EVP_PKEY_CTX_free(ctx);\n    return decrypted_data;\n}"
```

"instruction": "Generate a function that performs RSA public encryption using EVP_PKEY RSA key.",

```
"response": "unsigned char *public_encryption(EVP_PKEY *pkey, const unsigned char *plaintext, size_t\nplaintext_len, size_t *encrypted_len) {\n    EVP_PKEY_CTX *ctx = EVP_PKEY_CTX_new(pkey,\n    NULL);\n\n    if (ctx == NULL) {\n        // Handle error\n        return NULL;\n    }\n\n    if\n    (EVP_PKEY_encrypt_init(ctx) <= 0) {\n        // Handle error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    if (EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_PADDING) <= 0) {\n        // Handle error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    size_t outlen;\n    if\n    (EVP_PKEY_encrypt(ctx, NULL, &outlen, plaintext, plaintext_len) <= 0) {\n        // Handle error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    unsigned char *encrypted_data =\n    malloc(outlen);\n    if (encrypted_data == NULL) {\n        // Handle memory allocation error\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    if (EVP_PKEY_encrypt(ctx, encrypted_data,\n    &outlen, plaintext, plaintext_len) <= 0) {\n        // Handle error\n        free(encrypted_data);\n        EVP_PKEY_CTX_free(ctx);\n        return NULL;\n    }\n\n    *encrypted_len = outlen;\n    EVP_PKEY_CTX_free(ctx);\n    return encrypted_data;\n}"
```